

BoxAdmin — Fase 2: Motor de recurrencia

Especificación técnica para agente de desarrollo (Claude Code)

Septiembre 2026

Contexto

Sobre la base de la Fase 1 (Salas, Usuarios, Packs, Turnos, Reservas manuales), esta fase agrega el mecanismo que en TurnoFit resuelve “Rutina” + “Armado del mes siguiente”: patrones semanales fijos por alumno, generación automática de turnos futuros a partir de esos patrones, y una pantalla de detección de conflictos antes de publicar un mes.

Objetivo de esta fase: un admin carga la rutina fija de un alumno una sola vez (ej. “martes y jueves 18:00 en Pilates”) y el sistema genera solo los meses futuros automáticamente, mostrando conflictos (cupos llenos, alumno fuera de pack, vacaciones) antes de confirmar.

Criterio de éxito: cargar 10 rutinas fijas y correr la generación del mes siguiente produce las reservas correctas sin intervención manual, y el sistema detecta y lista claramente los casos que requieren revisión humana.

1. Dependencias adicionales

Ninguna librería nueva — se reutiliza BullMQ (ya configurado en Fase 0) para correr la generación como job en background en vez de bloquear el request HTTP (la generación de un mes completo para un salón con muchos alumnos puede tardar varios segundos).

ya instalado en fase 0: bullmq, ioredis, @nestjs/bullmq

2. Modelo de datos de esta fase (Prisma)

```
model RutinaFija {
  id          String    @id @default(cuid())
  tenantId    String
  tenant      Tenant    @relation(fields: [tenantId], references: [id])
  perfilId    String
  perfil      Perfil    @relation(fields: [perfilId], references: [id])
  salaId      String
  sala        Sala      @relation(fields: [salaId], references: [id])

  diaSemana  Int        // 0 = domingo ... 6 = sábado
  horaInicio String      // "18:00"
  horaFin     String

  activa      Boolean    @default(true)
  desde       DateTime   @db.Date // a partir de qué fecha aplica
  hasta       DateTime?  @db.Date // null = indefinida
```

```

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    @@index([tenantId, perfilId])
    @@index([tenantId, salaId, diaSemanas])
    @@map("rutinas_fijas")
}

enum EstadoMes {
    BORRADOR
    HABILITADO
}

model MesCalendario {
    // Estado de publicación por sala + mes, como máquina de estados explícita
    id          String    @id @default(cuid())
    tenantId    String
    tenant      Tenant    @relation(fields: [tenantId], references: [id])
    salaId      String
    sala        Sala      @relation(fields: [salaId], references: [id])

    anio        Int
    mes          Int      // 1-12
    estado      EstadoMes @default(BORRADOR)
    publicadoEn DateTime?
    publicadoPor String?

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    @@unique([tenantId, salaId, anio, mes])
    @@map("meses_calendario")
}

model VacacionAlumno {
    id          String    @id @default(cuid())
    tenantId    String
    perfilId    String
    perfil      Perfil    @relation(fields: [perfilId], references: [id])

    desde       DateTime @db.Date
    hasta       DateTime @db.Date
    motivo      String?
    devuelveClase Boolean @default(true)

    createdAt DateTime @default(now())

    @@index([tenantId, perfilId])
    @@map("vacaciones_alumnos")
}

model Ausencia {
    // Cierre de sala/salón por feriado, evento, reforma, etc. (Fase 6 lo expone completo en
    ↪ Mantenimiento,
    // pero el motor de generación ya debe respetarlas desde esta fase)
    id          String    @id @default(cuid())
    tenantId    String
    salaId      String?   // null = todo el salón

```

```

desde      DateTime
hasta      DateTime
todoElDia  Boolean  @default(true)
recuperable Boolean  @default(true)
motivo     String?

createdAt DateTime @default(now())

@@index([tenantId, salaId])
@@map("ausencias")
}

```

Migración:

```
pnpm dlx prisma migrate dev --name fase2_motor_recurrencia
```

3. Estructura de módulos

```

src/
├── rutinas/
│   ├── rutinas.module.ts
│   ├── rutinas.controller.ts
│   ├── rutinas.service.ts
│   └── dto/
├── calendario/
│   ├── calendario.module.ts
│   ├── calendario.controller.ts
│   ├── calendario.service.ts      # previsualización + detección de conflictos
│   └── dto/
├── ausencias/
│   ├── ausencias.module.ts
│   ├── ausencias.controller.ts
│   └── ausencias.service.ts
├── jobs/
│   └── generacion-mes/
│       ├── generacion-mes.processor.ts    # BullMQ worker
│       └── generacion-mes.service.ts      # lógica pura, testeable sin cola

```

4. Endpoints de esta fase

4.1 Rutinas fijas

Método	Ruta
POST	/rutinas
GET	/rutinas?perfilId=&salaId=
PATCH	/rutinas/:id
DELETE	/rutinas/:id

4.2 Generación del mes

Método	Ruta
POST	/calendario/:salaId/:anio/:mes/previsualizar
GET	/calendario/:salaId/:anio/:mes/conflictos
POST	/calendario/:salaId/:anio/:mes/publicar
GET	/calendario/:salaId/:anio/:mes

4.3 Vacaciones y ausencias

Método	Ruta
POST	/vacaciones-alumnos
GET	/vacaciones-alumnos?perfilId=
POST	/ausencias
GET	/ausencias?salaId=&desde=&hasta=

5. Lógica del motor de generación (generacion-mes.service.ts)

Algoritmo, en este orden:

1. Traer todas las RutinaFija activas de la salaId dada, vigentes en el rango del mes (desde `<= fin_de_mes` y (hasta es null o hasta `>= inicio_de_mes`)).
2. Para cada rutina, calcular todas las fechas del mes que caen en su díaSemana.
3. Descontar las fechas cubiertas por una Ausencia de esa sala/salón, y las cubiertas por una VacacionAlumno del perfil dueño de la rutina (esta última no cancela el turno para otros alumnos, solo excluye a ese alumno puntual).
4. Para cada fecha resultante: si no existe ya un Turno para esa sala+fecha+hora, marcarlo como “a crear”; si ya existe, verificar si el perfil ya tiene una Reserva ahí (idempotencia — la generación debe poder correrse más de una vez sin duplicar).
5. Antes de crear la reserva, verificar cupo disponible en el turno. Si no hay cupo → listar como **conflicto de tipo CUPO_LLENO**, no crear la reserva, no fallar el resto del proceso.
6. Verificar que el perfil esté dentro de la vigencia de su pack (Perfil.vigenciaHasta `>= fecha`) y tenga clases disponibles según el conteo del pack. Si no → conflicto de tipo FUERA_DE_PACK, no crear la reserva.
7. Acumular todos los conflictos en una estructura de retorno; **nunca abortar toda la generación por un conflicto individual** — el objetivo es que el admin revise una lista acotada de casos, no que se bloquee todo el mes por un alumno con el pack vencido.
8. En modo previsualizar: devolver el resumen sin tocar la base. En modo publicar: ejecutar todo dentro de una transacción por lote (o por sala) y, al finalizar, actualizar MesCalendario.estado = HABILITADO, publicadoEn, publicadoPor.

```
interface ResultadoGeneracionMes {
  turnosACrear: number;
  reservasACrear: number;
  conflictos: Array<{
    tipo: 'CUPO_LLENO' | 'FUERA_DE_PACK' | 'AUSENCIA_ALUMNO';
    perfilId: string;
    fecha: string;
    detalle: string;
  }>;
}
```

6. Checklist de aceptación de la Fase 2

- ☐ Se puede cargar una rutina fija para un alumno (día de semana + hora + sala).
 - ☐ previsualizar de un mes devuelve la lista completa de turnos/reservas a crear sin escribir nada en la base.
 - ☐ Los conflictos de cupo lleno, pack vencido y vacaciones del alumno se detectan correctamente y no interrumpen el resto del proceso.
 - ☐ publicar ejecuta la generación real, es idempotente (correrlo dos veces no duplica reservas) y actualiza el estado del mes a HABILITADO.
 - ☐ Una Ausencia de salón/sala excluye correctamente esas fechas de la generación para todos los alumnos.
 - ☐ Una VacacionAlumno excluye solo las reservas de ese alumno puntual, sin afectar el turno para el resto.
 - ☐ El job de generación corre en background (BullMQ), no bloquea el request HTTP para salones con muchos alumnos.
 - ☐ Tests unitarios del algoritmo de generación cubren: caso sin conflictos, cupo lleno, pack vencido, ausencia de salón, vacación de alumno, y re-ejecución idempotente.
-

7. Qué NO entra en esta fase

Self-service del alumno (reservar/cancelar desde afuera), profesor vinculado al turno, pagos y comprobantes, notificaciones por email/push — llegan en las fases 3, 4 y 5. Acá el foco es exclusivamente el motor de recurrencia y la detección de conflictos.